

فنشرح الجزء الأول من Route و من بعد
 فنشره من سيشن 16 بالكام من Jonas

طبعا احنا لازم نكون فاكرين في الدور الـ Callback hell
 عشان نقدر نشوف الحل بتاعها وهو الـ Promise
 يعني قبل ما تقرأ عن الجزء ده و الافضل تقرأ أنت
 عشان تعرف المشكلة كانت انج
 Route Academy + Jonas

Promise

كنا احنا شغالين بـ Callback
 نشغل بـ Promise عشان كارتج الـ Callback hell
 فكرتجاهي الـ Then وانها بتارج الـ Promise
 الـ Promise بتاخد Function

```
function one() {  
  return new Promise(function(callback) {  
    console.log('one')  
  })  
}  
  
function three() {  
  return new Promise(function(callback) {  
    console.log('two')  
  })  
}  
  
function two() {  
  return new Promise(function(callback) {  
    console.log('two')  
  })  
}
```

one.then()

الكلمات

هتساك نفسك فيك الباراميتير اللي على الFunction
 فقولك انك بيكون جوه الPromise حاليا مش على ال
 Function وعلشان تعمل Then فمظبوط لازم بتكون
 تعمل الreturn
 Promise

```
one().then(1 11 => {return two(1)})  

    .then(11 => {return three(1)})
```

يبقى كده معانا ال Then لازم اعلمها
 Anonymous Function
 ال Promise

تعالى نضرب على ال Catch فونفهم مع بعض فمساك ال
 rejected resolver
 Then دي على

```
function one() {  

console.log("one")  

    return new Promise(function(resolved, rejected) {
```

```
    console.log("one")  

    let error = false  

    if (error == false) {  

        resolved()  

    } else { rejected() }
```

```
one().then(11 => {return two(1)})
```

✍

وفي برشون Promise.all هنا يكون عاوز اعمل حاجة بعد ما كل
 الى في Then خلصوا هي بتاخد Array
 Promise.all([One, Two, Three]).then(function() {
 Console.log("All done")})

وفي طريقة ثانية وهي

Promise.resolve()

هنا الى مطلوب يرجع
 فتكلم عنها في الاخر

return Promise.resolve

Arrow Function

API
 Call back بنفس نظامه

Function getPasta()
 return new Promise((resolved, rejected) => {
 let allRecipes = []

let myHttp = new XMLHttpRequest()

myHttp.open('Get', 'API')

myHttp.send()

myHttp.addEventListener('Load',

() => { Console.log("Pasta")

allRecipes = JSON.parse(myHttp.response)
 recipes

object
 الخطة دة على API والى على String وانا عاوز احوال
 object
 كده يعني اصح recipes وبيهاكلها

back
 (resolved) => {

هنا برشون
 readyState
 change
 واقفها
 Check
 Send


```
myHttp.addEventListner('error', function() {
  // ...
})
```

و نفس الكلاس بالظبط في باقي ال Functions سواء ال Pizza
Saled

```
getPizza() {
  return new Promise((resolved, rejected) => {
    let allRecipes = []
    let myHttp = new XMLHttpRequest()
    myHttp.open('GET', url)
    myHttp.send()
    myHttp.addEventListner('load', () => {
      allRecipes = JSON.parse(myHttp.response).recipes
    })
    resolved()
  })
}
```

وبعد كده نرف الكلاس ال error ونعمل الحوار
دو مع ال Salad برضو كده انا جيت ال API وعاوز ارجعهم
Then تعالى بقى نخش على اخر حاجة وعل ال Then
كده انا بعد ال return ال Promise ودي ثاني حاجة فظهرها

```
getPasta().then(() => { return getPizza() })
.then(() => { return getSalad() })
.catch(() => { console.log('error') })
```

كده دي قالت حاجة فظهرها اهم حاجة اني بعمل return
مفروض ان ال catch لو وصل اى Error في التلافة فيبقى نفس ال
Promise ال return

وال ^{Resolved} ~~Rejected~~ تتعرف في ال catch طبعا وكلهم يطلعوا نفس

ال catch او ال error بمعنى اصح

فيه برفض ال ^{Final} ~~Final~~ ودي كبتها نفس ال catch ^{Then} لكن هي
هترجع الى مطلوب منها سواء كان فيه Error او لا
يعني بمعنى اصح هتتسغل في كل الحالات ولبتا ده مش الافضل
خالص ^{done Finish} ~~done Finish~~ ^{console.log} ~~console.log~~ ^{(1) = >} ~~(1) = >~~ ^{Final} ~~Final~~

Summary

ملخص لي قولنا كده احنا عملنا ال Promise عشان كارتق ال
تعمل Function ونخليها ^{return} ~~return~~ ^{new Promise} ~~new Promise~~ وندخلها ايتين ^{parameter} ~~parameter
resolved في حالة اللي جيلي بشكل مطلوب ^{reject} ~~reject~~ لو كانت اللي
جيلي فيه Error وبعد كده بنستخدم ال ^{XMLHttpRequest} ~~XMLHttpRequest~~ عشان اعندل
ال API وبعد كده ادخلها في load او تعمل ^{read} ~~read~~ ^{state change} ~~state change وتعمل if condition لو~~~~

كانت ب ⁴ ~~4~~ عشان تتأكد ان ^{Data} ~~Data~~ رجعت
وبعد كده تحط ال ^{resolved} ~~resolved~~ وكده علفكره عاملين بقى
فكرة ال Callback بس على ال ^{Promise} ~~Promise عشان منستخلص
Call Back hell ونستغل ^{Catch, Then} ~~Catch, Then~~ و من بعدها تستغل
على ^{addEvent} ~~addEvent~~ في ال ^{error} ~~error~~ ^{Listener} ~~Listener~~~~

كان يحصل في الفترة ماسه 2015:2017

Then Chaining ^{representation} ^{فانت فيط} ^{Console} ^{لاستفسر} ^{وما} ^{بوعملت} ^{response} ^{return} ^{response} ^{JSON}

Fetch('API').then((response) => response.json().then(data => console.log(data)))

فكره انت لازم تعملها Then ثاني
انت كد ال Then الاولى بتعامل مع ال response الى جاي من السيرفر وال Then الثانية بتعامل مع ال JSON بعد تحويله الى Object
الاولى بتستغل على ال response من JSON اما الثانية بتستغل على تحويل ال JSON الى Object

في الاخر اتعمل Async ^{await} بتقولها شيل تستي لحد ما الوقت يخلص (٢٠)

بس على شرطين:
[1] انك مش هتضع تعمل await غير لما return لازم انها ترجع Promise
[2] لازم تصطلي Function ^{Async}

function get Pizza() {
let response = await fetch('API')
let data = await response.json()
return data
}

پس بی حاشیه فنتلر اچدر حالیا پس ال
Variable لما يتحول الشجرة بتحتها لو حيتا ال Console
Fetch
هتكون Promise Pending
عاوزة لكن فيش اي راتا رجعت هتلاقي response
هناليد على نجاح السيرفر انه يرجع البيانات
Promise Status: Full Filled

Pending: لمسة مفيش نتيجة لسة شغال
 2) settled state: يعني خلاص العملية لسة سواء
 Full Filed: نزلنا

Full Filed: نجاح
Rejected: فشل
مثل مفيش نت

و بيقولات في نوعيه من ال Promise :

new Promise : وعدة
بأن تفعل
a Promise : وعدة

Consuming: آنکه استفاده‌ی حایات بترجیح promise را می‌کند
 SWR, Redux, (React) reactquery, Axios, Fetch

Consuming Promises (V264):

اولا كده علشان دي حلاوة اننا تسمح من كذا آخذ ال API
الى مستخدموها وفيها فكرة ان عليها ال name فنتشان كده
هو عمل Parameter علشان نستقبل ال name الى على API
بص هو اهم حاجة هنا انك ان ال Data منليكش عطلول
لان علشان نشوف البيانات الحقيقية الي موجودة في ال API
بتعمل مذكر. وهي اصلا اصلا بترجع Promise فكله احنا لازم
نعمل Then تانيك علشان ال Data
ده افضل واسهل كود:

```
const getCountryData = (country) => {  
  fetch(`API / ${country}`).then(  
    response => response.json()  
  ).then(  
    data => data.render(data[0])  
  )  
}
```

بالشكل
ده علشان سطر واحد
w function
بكتب

V265 Chaining Promises

دي بتعني اننا بتخليك تعمل كذا Then علشان ال return
ال Then الاولى بيكون بترجع Promise في است كده محتاج تيبا
Then تانيك علشان ترجع ال اكاملة في احنا كده بامكان لو احنا
عاوزيه نطهر Data من API و احنا كده بتعمل Then
وده مفروض بيدل ال Callback وده طبعا كان بيحصل قبل
hell
ال await / Async

```

const getCountryData = function(country) {
  fetch(`API / ${country}`).then(response => response.json())
  .then(data => {renderCountry(data, 'neighbour')})
}

```

دی اجزیا کی توفیق
 ال HTML مع Java Script
 دی الی بظہر الیانات جوہ Country

V266 handle Errors in Promises (rejected)
 with Network Connection

احتیاجنا فیکل علی اللہ وھنروح علی انسکت و Net وختیہا
 offline علشان تجربہ ال reject فیکل و لا
 انامکن Handle کل ال errors عہ طریق ان و الاخر
 اعمل catch علشان امساک ال error الی فی Then

insert Adjacent text
const renderError = Fun

inner.html
هي بيلا لا
علشان هي الافضل منها

insert Adjacent text	insert Adjacent HTML	inner HTML
بيحافظ على المحتوى القديم	بيحافظ على ال القديم Content	مش يحافظ على القديم بيسمح الي كان موجود
<u>safety on HTML</u>	No	No
X	X	عندما مشكك Reflow & repaint

N267 / Throw new Error

بعض الطبيعي في العادي جدًا انه لو معان API ال Backend هيكون
Handle جزئية ال Error دي ~~وانت~~ وانت بس بتعطها في
ال Catch لكن لو ال API مش Handle جزئية ال Error
ف انت هتستخدم

Throw new Error
بعض يا معالج في العادي فاك لما قولت ان ال response بتعمل
represent فيها بيان عن ال API دي ولو عملناها
وقا console في انت كده هيبيلا مثلاً ال

بول حالة ال API
(Status, OK) →


```
Fetch('API/${Country}').then(
  res => console.log(res))
```

```
response => { console.log(response)
```

```
if (!response.ok) { status, ok out 50
```

Throw new Error('Country ^{True} not found') // لا يوجد

کدام معنای این الدوله دی مش موجوده
بس بظهور مال فی الزمانه

```
return response.json(); }
```

وفي الاخر كمان بعد catch يستقل على ال error ده عشان
يوجد ال rejected وفي الاخر بعد catch ال error بعد ال
اخضر خالص وهو ال catch ال ال Throw new Error
فيفضل على ال response الى بتجيب ال Data

يعيد rejected في الآخر يعمل Catch Error بعد ال

أخضع خالص وهو Catch إلى Throw new Error
هيقفل على الـ response إلى تجيب الـ Data

```
catch (err = 7)
```

renderError (Something went wrong
\$ {err.message})

هناك بعض قيم البيانات التي على الـ `Throw` أن يوفق لو دخلت في الـ `Catch` لو كنا نستخدم عملنا الـ `Throw` كنا نعلم أننا في الـ `Catch` `undefined` لأن الـ `Error` احنا مش نعرف فيه

renderError

V269 | Async Behind The scenes

احنا عارفين ان ال JavaScript بتشتغل بـ non Blocking
على multi Thread لكن Single Thread
وي جافا انو c#

قبل كده حتمه ال Callback queue
منه طريقين micro queue و macro queue
عشان ~~يتم~~ يمشيوا (او طبعا لو معاك صورة فلا ف انت هنا
الصورة هتروح لـ webAPI مش هتروح على ال main thread